

主管
领导
审核
签字

哈尔滨工业大学（深圳）2023 年夏季学期

计算机系统试题（A）

题 号	一	二	三	四	五	总分
得 分						
阅卷人						

考生须知：本次考试为闭卷考试，考试时间为 120 分钟，总分 100 分。

姓名

学号

班号

学院

密

封

线

本题得分 _____

一、单项选择题（每题 1 分，共 20 分）

- 计算机操作系统抽象表示时，(A) 是对处理器、主存和 I/O 设备的抽象表示。
A. 进程 B. 虚拟存储器 C. 文件 D. 指令集架构 (ISA)
- 当调用 malloc 这样的 C 标准库函数时，(B) 可以在运行时动态的扩展和收缩。
A. 栈 B. 堆 C. 共享库 D. 内核虚拟存储器
- 下列叙述正确的是 (D)
A. X86-64 指令 "mov \$-1,%eax" 会使 %rax 的值变成 0xffffffffffffff
B. 在一条指令执行期间，CPU 不会两次访问内存
C. CPU 不总是执行 CS::RIP 所指向的指令，例如遇到 call、ret 指令时
D. 一条 mov 指令不可以使用两个内存地址操作数
- 下列传送指令中，哪一条是正确的 (D)
A. movb \$0x105, (%ebx)
B. movl %rdi, %rax
C. movq %rdi, \$105
D. movl 4(%rsp), %eax
- 在 Y86-64 指令集体系结构中，程序员可见的状态不包括 (B)
A. 程序寄存器 B. 高速缓存 C. 条件码 D. 程序状态
- 关于局部性的描述，不正确的是 (C)
A. 循环通常具有很好的时间局部性
B. 循环通常具有很好的空间局部性
C. 数组通常具有很好的时间局部性
D. 数组通常具有很好的空间局部性
- C 程序执行到整数或浮点变量除以 0 可能发生 (D)
A. 显示除法溢出错直接退出
B. 程序不提示任何错误
C. 可由用户程序确定处理方法
D. 以上都可能

-
8. 动态内存分配时产生内部碎片的原因不包括 (D)
- A. 维护数据结构的开销
 - B. 满足对齐约束
 - C. 分配策略要求
 - D. 超出空闲块大小的分配请求
9. 虚拟内存发生缺页的时候, 正确的叙述是 (C)
- A. 当发生虚拟内存缺页的时候, 程序会直接退出。
 - B. 缺页产生的中断请求通常由 CPU 产生。
 - C. 当处理程序处理完缺页错误之后, 会重新执行引发缺页的命令。
 - D. 当一个程序先后重复执行两次的时候, 会在相同的指令位置产生缺页错误
10. 以下有关编程语言的叙述中, 正确的是(D)。
- A. 计算机能直接执行高级语言程序和汇编语言程序
 - B. 机器语言可以通过汇编过程变成汇编语言
 - C. 汇编语言比高级语言有更好的可读性
 - D. 汇编语言和机器语言都与计算机系统结构相关
11. 假定变量 i、f 的数据类型分别是 int、float。已知 i=12345, f=1.2345e3, 则在一个 32 位机器中执行下列表达式时, 结果为“假”的是(C)。
- A. $i == (\text{int})(\text{float})i$
 - B. $i == (\text{int})(\text{double})i$
 - C. $f == (\text{float})(\text{int})f$
 - D. $f == (\text{float})(\text{double})f$
12. 以下关系表达式, 结果为“真”的是(B)。
- A. $2147483647\text{U} > -2147483648$
 - B. $(\text{unsigned}) -1 > -2$
 - C. $-1 < 0\text{U}$
 - D. $2147483647 < (\text{int}) 2147483648\text{U}$
13. 链接时两个同名的强符号, 以哪种方式处理? (D)
- A. 链接时先出现的符号为准
 - B. 链接时后出现的符号为准
 - C. 任一个符号为准
 - D. 链接报错
14. 以下关于程序中链接“符号”的陈述, 错误的是 (B)
- A. 赋初值的非静态全局变量是全局强符号
 - B. 赋初值的静态全局变量是全局强符号
 - C. 未赋初值的非静态全局变量是全局弱符号
 - D. 未赋初值的静态全局变量是本地符号
15. 虚拟内存系统中的实际物理地址和虚拟地址之间的关系是 (C)
- A. 1 对 1
 - B. 多对 1
 - C. 1 对多
 - D. 多对多
-

姓名

学号

班号

学院

密

封

16. CPU 一次访存时, 访问了 L1、L2、L3Cache 所用地址 A1、A2、A3 的关系是 (B)

- A. $A1 > A2 > A3$
- B. $A1 = A2 = A3$
- C. $A1 < A2 < A3$
- D. $A1 = A2 < A3$

17. 已知下面的数据结构, 假设在 X86-64 环境下要求对齐, 这个结构总大小是多少个字节? 如果重新排列其中的字段, 最少可以达多少个字节? (A)

```
struct {  
    char a;  
    float *b;  
    long c;  
    short d;  
    int e; };
```

- A. 32, 24
- B. 24, 24
- C. 28, 26
- D. 32, 26

18. 关于信号的描述, 以下不正确的是哪一个? (C)

- A. 在任何时刻, 一种类型至多只会有一个待处理信号
- B. 信号既可以发送给一个进程, 也可以发送给一个进程组
- C. SIGTERM 和 SIGKILL 信号既不能被捕获, 也不能被忽略
- D. 当进程在前台运行时, 键入 Ctrl-C, 内核就会发送一个 SIGINT 信号给这个

前台进程

19. 下列系统 I/O 的说法中, 正确的是 (D)

- A. C 语言中的标准 I/O 函数在不同操作系统中的实现代码一样
- B. 对于同一个文件描述符, 混用 RIO 包中的 rio_readnb 和 rio_readn 两个函数不会造成问题
- C. C 语言中的标准 I/O 函数是异步线程安全的
- D. 使用 I/O 缓冲区可以减少系统调用的次数, 从而加快 I/O 的速度

20. 若将标准输出重定向到文本文件 file.txt, 错误的是 (C)

- A. 需要先打开重定位的目标文件“file.txt”
- B. 设“file.txt”对应的 fd 为 4, 内核调用 dup2(4, 1)函数实现描述符表项的复制
- C. 复制“file.txt”的打开文件表项, 并修正 fd 为 1 的描述符
- D. 修改“file.txt”的打开文件表项的引用计数

本题得分 _____

二、填空题 (每空 1 分, 共 15 分)

1. 假定编译器规定 int 和 short 型长度分别为 32 位和 16 位, 执行下列语句:
unsigned short x=65530; unsigned int y=x; 得到 y 的机器数为
_____ 0000FFFAH _____. (用 16 进制表示, 勿省略前导 0)
2. 在计算机的存储体系中, 速度最快的是_____寄存器_____。
3. TLB (翻译后备缓冲器) 俗称快表, 是_____页表_____的缓存。

-
4. 链接器经过____**符号解析**____和____**重定位**____两个阶段, 将可重定位目标文件生成可执行目标文件。
5. -0.1 采用 IEEE 754 单精度浮点数格式表示的结果是 **CDCCCCBD/BDCCCCCD**。(16 进制形式)
6. 若高速缓存的块大小为 64 字节, 向量 v 的元素为 int , 则对 v 的步长为 1 的应用的不命中率为____**1/16**____。
7. 下面是 Y86-64 中的一段汇编程序, 请指出 $\text{jne } t$ 指令之后执行的那条指令的地址是____**0x00b**____。(16 进制表示)
- ```
0x000: xorq %rax,%rax
0x002: jne t
...
0x019: t: irmovq $3, %rdx
0x023: irmovq $4, %rcx
0x02d: irmovq $5, %rdx
```
8. 对于一个磁盘, 其平均旋转速率是 15000 RPM, 平均寻道时间是 4ms, 单个磁道上平均扇区数量是 800, 则这个磁盘的平均访问时间是\_\_\_\_**6.005**\_\_\_\_ms。(结果保留 3 位小数)
9. 子程序运行结束会向父进程发送\_\_\_\_**SIGCHLD**\_\_\_\_信号。
10. 向指定进程发送信号的 linux 命令是\_\_\_\_**kill**\_\_\_\_。
11. 若  $p.o \rightarrow \text{libx.a} \rightarrow \text{liby.a}$  且  $\text{liby.a} \rightarrow \text{libx.a} \rightarrow p.o$  则最小链接命令行\_\_\_\_**gcc p.o libx.a liby.a libx.a**\_\_\_\_。
12. 进程创建函数  $\text{fork}$  执行后返回\_\_\_\_**2**\_\_\_\_次; 进程加载函数  $\text{execve}$ , 如调用成功则返回\_\_\_\_**0**\_\_\_\_次。
13. Intel 桌面 X86-64 CPU 采用\_\_\_\_**小**\_\_\_\_(填写“大”或“小”)端模式。

本题得分 \_\_\_\_\_

### 三、简答题 (15 分)

1. 简述 Y86-64 流水线 CPU 中的冒险的种类与处理方法。(5 分)

答: (3 分) 数据冒险: 指令使用寄存器  $R$  为目的, 瞬时之后使用  $R$  寄存器为源。

处理方法有暂停: 通过在执行阶段插入气泡 (bubble/nop), 使得当前指令执行暂停在译码阶段; 数据转发: 增加  $\text{valM/valE}$  的旁路路径, 直接送到译码阶段; 加载使用冒险: 指令暂停在取指和译码阶段, 在执行阶段插入气泡 (bubble/nop)。

(2 分) 控制冒险: 分支预测错误: 在条件为真的地址  $\text{target}$  处的两条指令分别插入 1 个 bubble。ret: 在 ret 后插入 3 个 bubble。

姓名

学号

班号

学院

密

封

2. 简述缓冲区溢出攻击的原理以及防范方法。(5分)

答: 攻击原理(3个采分点):

- (1) 程序输入缓冲区写入特定的数据, 例如在 gets 读入字符串时
- (2) 使位于栈中的缓冲区数据溢出, 用特定的内容覆盖栈中的内容, 例如函数返回地址等
- (3) 使得程序在读入字符串, 结束函数 gets 从栈中读取返回地址时, 错误地返回到特定的位置, 执行特定的代码, 达到攻击的目的。

防范方法(4个采分点, 每个 0.5 分):

- (1) 代码中避免溢出漏洞: 例如使用限制字符串长度的库函数。
- (2) 随机栈偏移: 程序启动后, 在栈中分配随机数量的空间, 将移动整个程序使用的栈空间地址。
- (3) 限制可执行代码的区域
- (4) 进行栈破坏检查——金丝雀

3. 结合 fork, execve 函数, 简述在 shell 中加载和运行 hello 程序的过程。(5分)

答: ① 在 shell 命令行中输入命令: ./hello

② shell 命令行解释器构造 argv 和 envp;

③ 调用 fork() 函数创建子进程, 其地址空间与 shell 父进程完全相同, 包括只读代码段、读写数据段、堆及用户栈等

④ 调用 execve() 函数在当前进程(新创建的子进程)的上下文中加载并运行 hello 程序。将 hello 中的 .text 节、.data 节、.bss 节等内容加载到当前进程的虚拟地址空间

⑤ 调用 hello 程序的 main() 函数, hello 程序开始在一个进程的上下文中运行。

四、系统分析题（15 分）

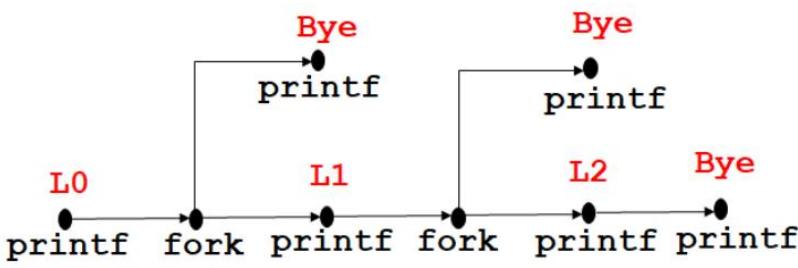
1. 有如下 C 语言程序，请画出程序的进程图，并给出所有可能的输出。（10 分）

```

int main()
{
 printf("L0\n");
 if(fork() != 0) {
 printf("L1\n");
 if(fork() != 0) {
 printf("L2\n");
 }
 }
 printf("Bye\n");
 return 0;
}

```

- (1) 画出进程图；（5 分）
- (2) 请写出可能的输出。（5 分）



每一各输出 1 分，共 5 分

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| L0  | L0  | L0  | L0  | L0  |
| L1  | L1  | L1  | Bye | Bye |
| L2  | Bye | Bye | L1  | L1  |
| Bye | L2  | Bye | L2  | Bye |
| Bye | Bye | L2  | Bye | L2  |
| Bye | Bye | Bye | Bye | Bye |

姓名

学号

班号

学院

密

封

2. 已知函数 arith 的汇编代码如左图所示, 请填写汇编代码对应的 C 语言代码缺失的部分①~⑤。(5 分)

```
arith:
xorq %rsi,%rdi
leaq (%rdi,%rdi,4),%rax
leaq (%rax,%rsi,2),%rax
subq %rdx,%rax
retq
```

```
long arith(long x, long y, long z){
 long t1 = ①_____ ;
 long t2 = ②_____ ;
 long t3 = ③_____ ;
 long t4 = ④_____ ;
 ⑤_____ ; }
```

(1)  $x^y$

(2)  $t1 + t1 \ll 2$  或  $5*t1$

(3)  $t2 + (y \ll 1)$

(4)  $t3 - z$

(5) return t4

每空一分

注意: 5t 和 2y 从 C 语言角度都是错的

本题得分 \_\_\_\_\_

## 五、综合设计题（35 分）

1. （15 分）为 Y86-64 CPU 增加一指令“iaddq V, rB”，将常量数值 V 加到寄存器 rB。参考 irmovq、OPq 指令，请设计 iaddq 指令在各阶段的微操作。

| 指令    | irmovq V,rB                                                                                                            | OPq rA, rB                                                                              | iaddq V,rB                                                                                                             |
|-------|------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| 取指    | icode:ifun $\leftarrow$ M1[PC]<br>rA:rB $\leftarrow$ M1[PC+1]<br>valC $\leftarrow$ M8[PC+2]<br>valP $\leftarrow$ PC+10 | icode:ifun $\leftarrow$ M1[PC]<br>rA:rB $\leftarrow$ M1[PC+1]<br>valP $\leftarrow$ PC+2 | icode:ifun $\leftarrow$ M1[PC]<br>rA:rB $\leftarrow$ M1[PC+1]<br>valC $\leftarrow$ M8[PC+2]<br>valP $\leftarrow$ PC+10 |
| 译码    | valB $\leftarrow$ 0                                                                                                    | valA $\leftarrow$ R[rA]<br>valB $\leftarrow$ R[rB]                                      | valB $\leftarrow$ R[rB]                                                                                                |
| 执行    | valE $\leftarrow$ valB+valC                                                                                            | valE $\leftarrow$ valB OP valA<br>Set CC                                                | valE $\leftarrow$ valB+valC<br>Set CC                                                                                  |
| 访存    |                                                                                                                        |                                                                                         |                                                                                                                        |
| 写回    | R[rB] $\leftarrow$ valE                                                                                                | R[rB] $\leftarrow$ valE                                                                 | R[rB] $\leftarrow$ valE                                                                                                |
| 更新 PC | PC $\leftarrow$ valP                                                                                                   | PC $\leftarrow$ valP                                                                    | PC $\leftarrow$ valP                                                                                                   |

评分标准：每个空格 3 分，酌情给分



姓名

学号

班号

学院

```
.....
2. (12 分) 向量元素和计算的相关程序如下, 请部分改写或重写计算函数
vector_sum, 进行速度优化, 并简要说明优化的依据。

/*向量的数据结构定义 */
typedef struct{
 int len; //向量长度, 即元素的个数
 float *data; //向量元素的存储地址
} vec;

/*获取向量长度*/
int vec_length(vec *v){return v->len;}

/* 获取向量中指定下标的元素值, 保存在指针参数 val 中*/
int get_vec_element(*vec v, size_t idx, float *val){
 if (idx >= v->len)
 return 0;
 *val = v->data[idx];
 return 1;
}

/*计算向量元素的和*/
void vector_sum(vec *v, float *sum){
 long int i;
 *sum = 0; //初始化为 0
 for (i = 0; i < vec_length(v); i++) {
 float val;
 get_vec_element(v, i, &val); //获取向量 v 中第 i 个元素的值, 存入 val 中
 *sum = *sum + val; //将 val 累加到 sum 中
 }
}
```

---

```
void vector_sum(vec *v, float *sum)
{
 long int i;
 *sum = 0;
 int length = vec_length(v);
 float *d = v->data;
 float acc = 0; //局部变量累计结果
 for (i = 0; i < length; i++)
 acc = acc + d[i]; //消除不必要的内存引用
 *sum = acc;
}
```

优化方法：

- (1) 代码移动，将 `vec_length()` 移到循环外。
- (2) 消除不必要的函数调用，将 `get_vec_element()` 转换为通过 `d[i]` 来获取每次循环的元素。
- (3) 用局部变量 `acc` 做累加，消除不必要的内存引用
- (4) 可做循环展开

评分标准：答对一种优化给 10 分，2 种及以上满分。

姓名

学号

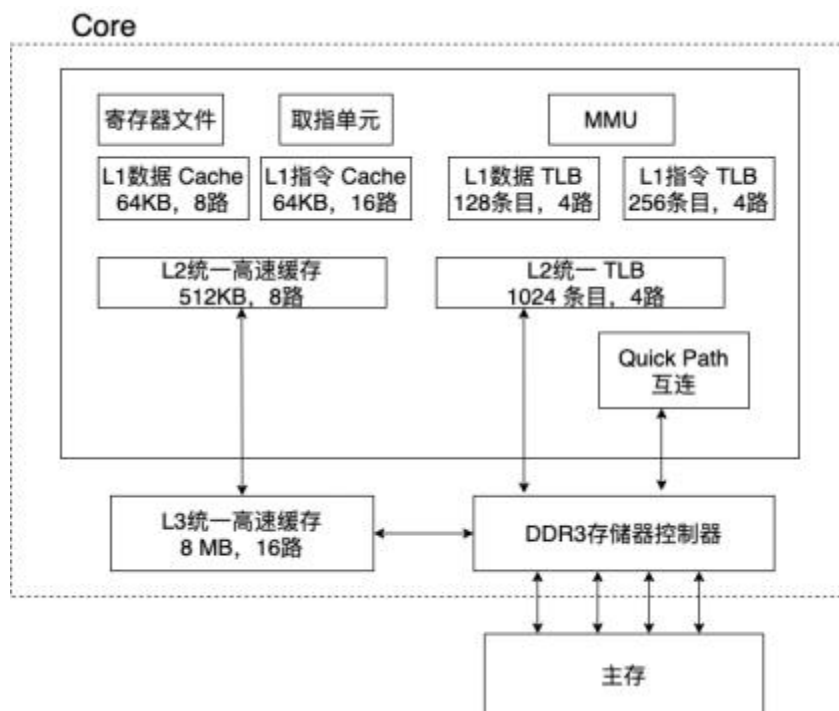
班号

学院

密

封

3. (8 分)某处理器的虚拟地址为 32 位。虚拟内存的页大小是 4KB,物理地址为 48 位,Cache 块大小为 32B。物理内存按照字节寻址。其内部结构如下图所示,依据这个结构,回答下面几个问题:



(1)L1 数据 Cache 有多少组? 相应的 Tag 位,组索引位和块内偏移位分别是多少?

答: 有 256 组, Tag 有 35 位, 组索引位 8 位, 块内偏移 5 位。

(2) 对于某数据, 其访问的虚拟地址为 0x829358B, 则该地址对应的 VPO 为多少? 对应的 L1 TLBI 位为多少? (用 16 进制表示)

答: 所以组索引 5 位, VPO 为 0x58B, 组索引对应的位为 0x13。

(3) 对于某指令, 其访问的物理地址为 0x829358B, 则该地址访问 L1 Cache 时, CT 位为多少? CO 位为多少? (用 16 进制表示)

答: CT 位为 0x8293 CO 位为 0x0B。

